

Functional Dependencies, Redundancy and Data Anomalies

Functional dependencies show how one attribute determines another attribute in a relation. In this E-Commerce Order Management System, the **Primary Key determines all other attributes** in its corresponding table.

1. Functional Dependencies

Customer

Customer_ID → Customer_Name, Email, Phone, Address, Registration_Date

Seller

Seller_ID → Seller_Name, Email, Phone, Address

Category

Category_ID → Category_Name

Product

Product_ID → Product_Name, Category_ID, Seller_ID,
Price, Stock_Quantity, Product_Status

Orders

Order_ID → Customer_ID, Order_Date, Order_Status, Total_Amount

Order_Details

Order_Detail_ID → Order_ID, Product_ID, Quantity,
Unit_Price, Subtotal

Payment

Payment_ID → Order_ID, Payment_Method, Amount,
Payment_Date, Payment_Status

Shipping

Shipping_ID → Order_ID, Shipping_Address,
Tracking_Number, Delivery_Status

These dependencies follow directly from the entities and attributes defined in the database design.

_2. Redundancy Analysis

The database design reduces redundancy by storing different types of information in separate tables.

For example:

- **Customer information** is stored only in the Customer table.
- **Seller information** is stored only in the Seller table.
- **Category details** are separated from the Product table.
- A product is not repeatedly stored for every order; instead, Order_Details connects Orders and Product.
- Payment and shipping information are maintained separately from order information.

The Order_Details table is especially important because it connects orders and products without unnecessarily repeating complete product information.

3. Data Anomalies

If the database is not properly normalized, the following anomalies may occur:

a) Insertion Anomaly

An insertion anomaly occurs when data cannot be inserted without adding unnecessary information.

Example:

If product and category information were stored together with order details, a new product might not be added unless an order exists for that product.

Solution:

The Product and Category tables are maintained separately.

b) Update Anomaly

An update anomaly occurs when the same information is stored in multiple places and must be updated repeatedly.

Example:

If a seller's phone number were stored with every product, changing the phone number would require updating multiple records.

Solution:

Seller information is stored only once in the Seller table, and products use Seller_ID as a foreign key.

c) Deletion Anomaly

A deletion anomaly occurs when deleting one record unintentionally removes other important information.

Example:

If the last order containing a particular product is deleted from a combined table, the product information could also be lost.

Solution:

Product information is stored independently in the Product table, while order information is maintained in Orders and Order_Details.

Conclusion

The proposed database design minimizes redundancy and reduces insertion, update, and deletion anomalies by separating customer, seller, category, product, order, payment, and shipping information into individual relations. Primary keys uniquely identify records, while foreign keys maintain relationships between tables. Overall, the design provides a structured and normalized foundation for the E-Commerce Order Management System.